COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	September 17, 2026	**DATE SUBMITTED:**	September 22, 2026
NAME:	Uchi, Mao Sophie S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 **Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of shell, merge and quick sort.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```
11      int unsortedArr[max_size];
12      srand(time(0));
13      for(int i = 0; i < max_size; i++){
14          unsortedArr[i] = rand()%100;
15      }
16
17      std::cout << "Unsorted Array:\n";
18      displayArr(unsortedArr);
```

```
29  void displayArr(int arr[]){
30      for(int i = 0; i < max_size; i++){
31          std::cout << arr[i] << " ";
32      }
33      std::cout << std::endl;
34  }
```

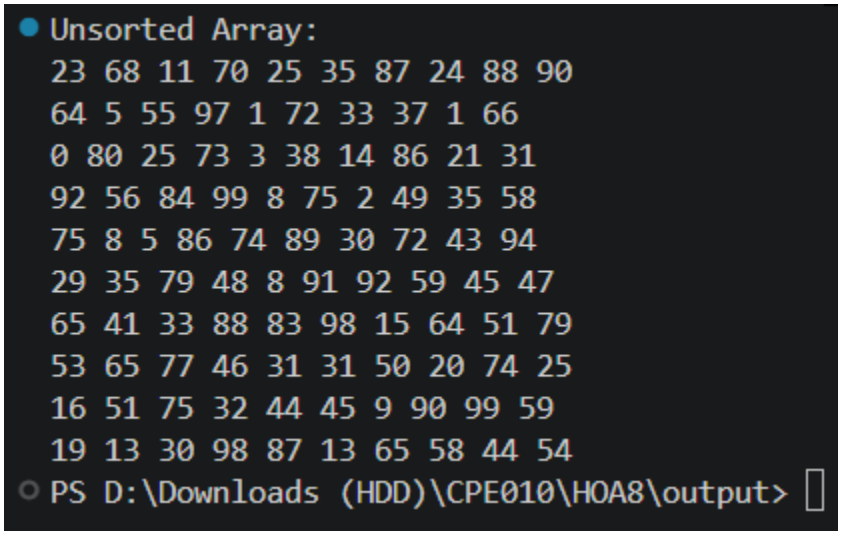
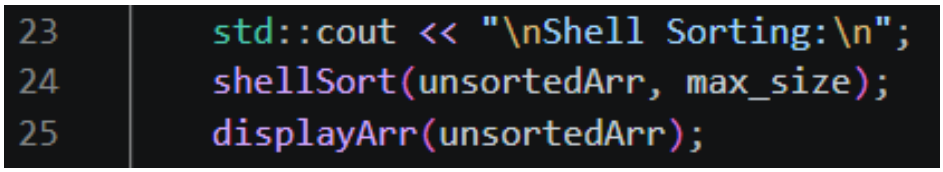
	
Observations	From our previous activity, we had the same procedure of generating random integers as a base for our search algorithms. This array has a size of 100, containing elements from 1-100, using the random modulo operation and should be sorted by the end of each test using Shell Sort, Merge Sort and Quick Sort. The algorithms can output different integers, but the main objective is as long as the numbers are in numerical ascending order.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

```cpp
template <typename T>
void shellSort(T data[], int max_size) {

    for (int gap = max_size/2; gap > 0; gap /=2) {
        for (int i = gap; i < max_size; i++) {
            T temp = data[i];
            int j = i;
            while (j >= gap && data[j - gap] > temp) {
                data[j] = data[j - gap];
                j -= gap;
            }
            data[j] = temp;
        }
    }
}
```

```
Shell Sorting:
0 0 0 1 1 1 4 5 6 6 7 8 9 9 9 10 14 16 16 18 18 19 19 22
22 27 28 30 31 31 33 35 38 39 39 40 41 42 42 43 44 45 45
45 46 46 53 54 55 55 56 57 57 60 61 62 63 63 64 65 66 66
66 66 67 68 72 72 72 73 74 75 76 77 77 79 79 80 81 83 84
86 87 88 89 89 89 90 91 91 91 93 94 95 95 96 96 97 98 99
PS D:\Downloads (HDD)\CPE010\HOA8>
```

Observations

Similar to insertion sort, instead of comparing current elements from the previous elements individually and linearly, we will be introducing Gap into our code. What is the gap for? From the previous activity, without Gap from insertion sort, the time complexity would be at o(n) as it will rely on the unsorted elements to be in ascending order; if the unsorted elements are in descending order, every comparison will start from left to right until the elements are sorted. With Gap, the insertion sort will start at the left and middle index, which is half of the nth array size; both are compared to the right until the middle index is currently at the right of the array. We must start over with a smaller Gap, a Gap of 2 from a 10 array size as our example, we compare the 5 elements and move on to the other 5 elements to compare again. We must start again with an even smaller Gap, a gap of 1, which is what insertion sort do; from our previous comparisons, it is much easier to compare them as the elements that need to be in their positions are closer, with that, linearly compare the elements until they're sorted.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot

```
21  template <typename T>
22  void merge(T data[], int left, int mid, int right) {
23
24      int sizeL = mid-left + 1;
25      int sizeR = right - mid;
26      T *L = new T[sizeL];
27      T *R = new T[sizeR];
28      for (int i = 0; i < sizeL; i++){
29          L[i] = data[left + i];
30      }
31      for (int j = 0; j < sizeR; j++){
32          R[j] = data[mid + 1 + j];
33      }
34      int i = 0, j = 0, k = left;
35      while (i < sizeL && j < sizeR) {
36          if (L[i] <= R[j]) {
37              data[k] = L[i++];
38              i++;
39          } else {
40              data[k] = R[j++];
41              j++;
42          }
43          k++;
44      }
45      while (i < sizeL) {
46          data[k++] = L[i++];
47      }
48      while (j < sizeR) {
49          data[k++] = R[j++];
50      }
51  }
```

```
27      std::cout << "\nMerge Sorting:\n";
28      mergeSort(unsortedArr, 0, max_size-1);
29      displayArr(unsortedArr);
```

```
Merge Sorting:
0 0 1 3 5 6 8 9 10 11 11 12 14 14 15 17 17 21 24 25 27 28 28 29 31 31 31
 32 32 33 37 37 41 41 42 43 43 44 45 46 47 50 50 51 52 52 52 52 52 53 53
 54 54 57 58 59 66 68 69 70 70 71 72 72 73 73 73 74 74 75 75 76 77 78 79
 79 80 82 83 84 85 86 87 87 89 89 90 91 91 92 93 93 94 94 95 95 95 96 98
 98
PS D:\Downloads (HDD)\CPE010\HOA8>
```

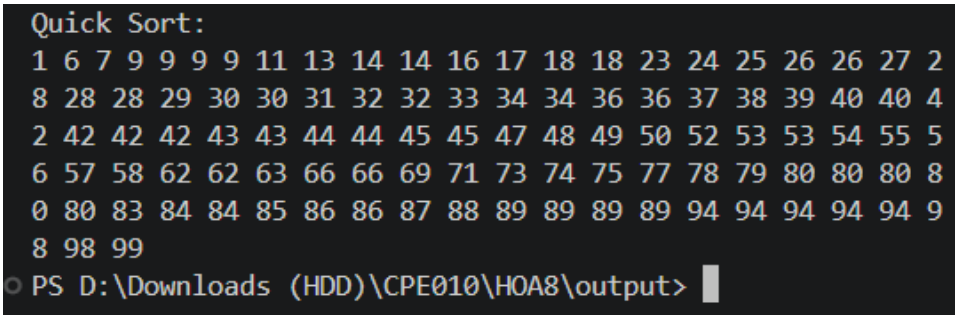
Observations

By the name itself, the elements merge until the elements sorts into numerical ascending order. Testing with 100 elements, we must divide the array into subsets, portion the size into 2, until every subsets are individual elements and in no way to continue dividing. Compare the individual elements to their previous subset and merge. Continue to merge each subsets until the large subsets become a whole sorted array.

Table 8-4. Quick Sort Algorithm

Code + Console Screenshot

```
64  template<typename T>
65  int partition(T data[], int left, int right) {
66
67      T pivot = data[right];
68      int i = left - 1;
69      for (int j = left; j <= right - 1; j++) {
70          if (data[j] < pivot) {
71              i++;
72              std::swap(data[i], data[j]);
73          }
74      }
75      std::swap(data[i + 1], data[right]);
76      return i + 1;
77  }
78
79  template<typename T>
80  void quickSort(T data[], int left, int right) {
81
82      if (low < high) {
83          int pivot = partition(data, left, right);
84          quickSort(data, left, pivot - 1);
85          quickSort(data, pivot + 1, right);
86      }
87  }
```

	
Observations	*As much as I want a "quick" observation, this sorting algorithm is hard for me to summarize.* Before comparing happens, we must introduce Pivot into our code and what does it do? It is our basis on which elements are greater or lesser, if the comparing element is greater than the pivot, it goes in the right partition; if comparing element is lesser than that pivot, it goes in the left partition. What is Partition, and why is it stated from our previous statement; it is a subarray that separates the greater and lesser values that is compared to the pivot. First, we choose our pivot, and for me, we should select the last index of the array; this will be our basis for comparison. Compare the pivot and the remaining elements linearly until the pivot is greater than the element. After comparing the first iteration, we must find another pivot to compare; we use the last index of the left partition(smaller subset), continue until every large element is at the right partition and the left partition has none elements left to compare. We move on to the right partition and simply do the same thing on what we've done with the left partition; iterate until all partitions are sorted.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

For me, the answer will be yes, as handling every single division with recursion stops the quick sort algorithm when the subset gets smaller and uses insertion sort for an iterative algorithm.
Demonstrating my answer, we will have an example [98, 76, 54, 32, 10] as our unsorted array, choosing the 3rd index or the element [54] as our pivot; we can distribute each smaller or larger elements to their designated partitions, [98, 76] will be at the right partition and [32, 10] will be at the left partition. This is where we can replace Quick Sorting with Insertion Sorting, as the pivot is in its sorted index, we will compare all the elements inside the left and right partition; with sorted left partition [10, 32] and sorted right partition [76, 98]. All you had to do is to combine two subsets and pivot, with a sorted array looking like this [10, 32, 54, 76, 98].

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance?
I'd choose the Insertion sorting algorithm as the array example provided has a size of 15, which is small compared to our testing examples at our procedure activity, it is much quicker as insertion sort uses a simple looping sort. It relies on the sorted order whether comparing each element is needed as its best case scenario.
Why can merge sort and quick sort have O(N • log N) for their time complexity?
These sorts having the same time complexity as both follow the Divide and Conquer approach and merge the subsets together as sorted. [LogN] indicates the number of times the array is divided in half until there are no more values to be divided; [N] indicates the accomplished work of comparison, division and reassembling the values. [NlogN] implies that [LogN] carries out by multiplying the values by [N] times.

Conclusion
My conclusion for this activity is I've learned more about sorting algorithms, previously with bubble sort, selection sort and insertion sort; now with 3 additional sorting algorithms with Shell sort, merge sort and quick sort. Shell Sort discusses the use of Gap in a modified insertion sort, improving its efficiency by allowing comparisons between elements that are farther apart, particularly when elements are in descending order. It explains starting with a larger Gap and reducing it gradually to 1, facilitating easier sorting as elements become closer to their correct positions. Merge Sort describes the merging process in sorting, dividing an array into smaller subsets until individual elements are formed and subsequently merging them into a sorted array. Quick Sort introduces the concept of Pivot in partitioning, where elements are compared against a chosen pivot to sort them into left and right partitions. The process continues iteratively until all elements are sorted within their respective partitions. With this newly acquired knowledge of sorting algorithms, it will definitely be easier to sort elements, different advantages to use each algorithm depending on whether what we seek from our sorting methods are time complexity, which sorting methods have the best, average and worst case performance.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Summarization of my observations**

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.